

Fundamentos de Sistemas Robóticos com ROS 2

Etapa 5

Professor Mario Pego

mario.peggo@prof.infnet.edu.br



Cronograma



Fundamentos de Sistemas Robóticos com ROS 2

<i>Etapas</i>	<i>Segunda-feira/Quarta-feira</i>
Etapa 1	20/07/26 e 22/07/26
Etapa 2/Leitura TP1	27/07/26 e 29/07/26
Etapa 3	03/08/26 e 05/08/26
Etapa 4/Entrega do TP1/Leitura TP2	10/08/26 e 12/08/2026
Etapa 5	17/08/26 e 19/08/26
Etapa 6/Entrega TP2/Leitura do TP3	24/08/26 e 26/08/26
Etapa 7	31/08/26 e 02/09/26
Etapa 8/Entrega TP3	07/09/26 (FERIADO) e 09/09/26
Etapa 9/Leitura do AT	14/09/26 e 16/09/26
Entrega Assessment	21/09/26 e 23/09/26
Reentrega	28/09/26 e 30/09/26

Implementar chamadas
síncronas entre nós com
serviços Client/Server ROS 2

O que vamos ver

1

Revisão: serviços Client/Server

Request/response, o que já sabemos desde a Etapa 3/4

2

Estado interno e concorrência

Por que um nó com serviço + timer + subscriber precisa de cuidado

3

threading.Lock

Protegendo um atributo compartilhado entre callbacks

4

MultiThreadedExecutor

Rodando múltiplos callbacks em paralelo com segurança

5

controlador.py

O nó completo do TP3 Exercício 2

Relembrando: serviços Client/Server

Da Etapa 3 e 4 — comunicação síncrona um-para-um

- Um client envia uma requisição e aguarda uma resposta de um server — foi assim que construímos modo_server/modo_client no TP2
- std_srvs/srv/SetBool: request com um campo bool, response com success + message
- Hoje o serviço fica mais interessante: em vez de só responder, ele vai mudar o estado interno do próprio nó

Um nó que decide sozinho

De 'responder a um pedido' para 'gerenciar um estado ao longo do tempo'

- No TP2, modo_server só respondia True/False — não guardava nada entre chamadas
- No TP3, o controlador.py guarda um estado (parado / movendo / emergencia) que persiste entre chamadas de serviço
- Esse estado também pode mudar sozinho — se a bateria cair, o nó decide entrar em emergência sem que ninguém peça
- É exatamente esse tipo de nó 'autônomo' que aparece em sistemas robóticos reais

A máquina de estados do controlador

Três estados possíveis, definidos pelo TP2 Exercício 2



A transição para emergencia é automática (via /bateria) e não tem volta pelo serviço — só reiniciando o nó, como pede o TP3.

Três fontes de eventos, um só estado

Serviço, timer e subscriber podem disparar 'ao mesmo tempo'

- O callback do serviço /mudar_estado pode escrever em self.estado
- O callback do subscriber /bateria também pode escrever em self.estado (quando detecta bateria crítica)
- O timer que publica /cmd_vel Lê self.estado a cada 0.5s
- Com MultiThreadedExecutor, esses três callbacks podem rodar em threads diferentes — se dois escreverem ao mesmo tempo, o estado final é imprevisível

Condição de corrida: o que pode dar errado

Duas threads escrevendo no mesmo atributo, sem proteção

```
# SEM proteção — NÃO faça isso
def mudar_estado_callback(self, req, resp):
    self.estado = 'movendo' if req.data \
        else 'parado'
    return resp

def bateria_callback(self, msg):
    if msg.data < 20.0:
        self.estado = 'emergencia'
```

O risco

Se o serviço e o callback de bateria rodarem ao mesmo tempo em threads diferentes, uma escrita pode 'vencer' a outra de forma imprevisível — o robô pode continuar em 'movendo' mesmo com bateria crítica.

threading.Lock: a seção crítica

Só uma thread por vez pode estar 'dentro' do lock

```
import threading

self.lock = threading.Lock()

with self.lock:
    self.estado = novo_valor
    # nenhuma outra thread entra aqui
    # até este bloco terminar
```

with self.lock

O 'with' garante que o lock é liberado mesmo se uma exceção acontecer dentro do bloco. Toda leitura E escrita de self.estado deve ficar dentro de um 'with self.lock'.

Anatomia do controlador

`__init__` com estado, lock, serviço, subscriber e publisher

salvar em `comunicacao_ros2/comunicacao_ros2/controlador.py`

```
class Controlador(Node):  
    def __init__(self):  
        super().__init__('controlador')  
        self.lock = threading.Lock()  
        self.estado = 'parado'  
        self.srv = self.create_service(  
            SetBool, 'mudar_estado',  
            self.mudar_estado_callback)
```

Estado inicial

O robô sempre começa 'parado'. O nome do serviço é 'mudar_estado' — sem a barra inicial, o `create_service` já registra em `/mudar_estado`.

O callback do serviço

True inicia movimento, False para o robô

salvar em `comunicacao_ros2/comunicacao_ros2/controlador.py`

```
def mudar_estado_callback(self, request, response):  
    with self.lock:  
        if self.estado != 'emergencia':  
            self.estado = (  
                'movendo' if request.data  
                else 'parado')  
            response.success = True  
            response.message = (  
                f'Estado alterado para {self.estado}')        self.get_logger().info(response.message)  
    return response
```

Por que checar emergencia

Uma vez em emergencia, o serviço não deve conseguir tirar o robô desse estado — é uma trava de segurança pedida implicitamente pelo TP3 (só reinicia o nó).

Publicando /cmd_vel continuamente

Um timer lê o estado e decide a velocidade

salvar em `comunicacao_ros2/comunicacao_ros2/controlador.py`

```
self.pub_cmd = self.create_publisher(  
    Twist, '/cmd_vel', 10)  
self.create_timer(0.5, self.publicar_cmd_vel)  
  
def publicar_cmd_vel(self):  
    with self.lock:  
        estado = self.estado  
        msg = Twist()  
        msg.linear.x = 0.5 if estado == 'movendo' else 0.0  
        self.pub_cmd.publish(msg)
```

EXEMPLO — NÃO EXECUTAR AGORA

Copiar antes de usar

Note que lemos `self.estado` para uma variável local DENTRO do lock, e só publicamos DEPOIS de liberar o lock — isso minimiza o tempo que a seção crítica fica travada.

Monitorando /bateria

Transição automática para emergencia

salvar em `comunicacao_ros2/comunicacao_ros2/controlador.py`

```
self.sub_bateria = self.create_subscription(
    Float32, '/bateria',
    self.bateria_callback, 10)

def bateria_callback(self, msg):
    with self.lock:
        if msg.data < 20.0 and \
            self.estado != 'emergencia':
            self.estado = 'emergencia'
            self.get_logger().warn(
                'BATERIA CRÍTICA: emergencia')
```

Quem publica /bateria?

O controlador só assina — quem publica de verdade é o `estado_robo.py`, que construímos na Competência 2. Sem ele rodando, este callback nunca é chamado.

controlador.py — parte 1 de 3

Imports e construtor: estado, lock, serviço, subscriber e publisher

salvar em `comunicacao_ros2/comunicacao_ros2/controlador.py`

```
import threading
import rclpy
from rclpy.node import Node
from rclpy.executors import MultiThreadedExecutor
from std_srvs.srv import SetBool
from std_msgs.msg import Float32
from geometry_msgs.msg import Twist
class Controlador(Node):
    def __init__(self):
        super().__init__('controlador')
        self.lock = threading.Lock()
        self.estado = 'parado'
        self.srv = self.create_service(SetBool, 'mudar_estado', self.mudar_estado_callback)
        self.sub_bateria = self.create_subscription(Float32, '/bateria', self.bateria_callback, 10)
        self.pub_cmd = self.create_publisher(Twist, '/cmd_vel', 10)
        self.create_timer(0.5, self.publicar_cmd_vel)
        self.get_logger().info('nó iniciado')
```

controlador.py — parte 2 de 3

Os dois callbacks: serviço e bateria

salvar em `comunicacao_ros2/comunicacao_ros2/controlador.py`

```
def mudar_estado_callback(self, request, response):
    with self.lock:
        if self.estado != 'emergencia':
            self.estado = 'movendo' if request.data else 'parado'
            response.success = True
            response.message = f'Estado alterado para {self.estado}'
        self.get_logger().info(response.message)
    return response

def bateria_callback(self, msg):
    with self.lock:
        if msg.data < 20.0 and self.estado != 'emergencia':
            self.estado = 'emergencia'
            self.get_logger().warn('BATERIA CRÍTICA: emergencia')
```


controlador.py — parte 3 de 3

Publicação de /cmd_vel e o main() com MultiThreadedExecutor

salvar em `comunicacao_ros2/comunicacao_ros2/controlador.py`

```
def publicar_cmd_vel(self):
    with self.lock:
        estado = self.estado

    msg = Twist()
    msg.linear.x = 0.5 if estado == 'movendo' else 0.0
    self.pub_cmd.publish(msg)

def main(args=None):
    rclpy.init(args=args)
    node = Controlador()
    executor = MultiThreadedExecutor()
    executor.add_node(node)
    try:
        executor.spin()
    finally:
        executor.shutdown()
        node.destroy_node()
        rclpy.shutdown()
```

Por que `MultiThreadedExecutor` aqui é obrigatório

Situação	<code>SingleThreadedExecutor</code>	<code>MultiThreadedExecutor</code>
Serviço + timer + subscriber no mesmo nó	Um fica esperando o outro terminar	Os três podem rodar em paralelo
Chamar /mudar_estado enquanto bateria cai	A resposta do serviço pode atrasar	Ambos processados quase simultaneamente
Precisa de <code>threading.Lock</code> ?	Não (só uma thread por vez)	Sim — sempre que houver estado compartilhado

Registrando o `entry_point`

controlador precisa entrar no `setup.py`

salvar em `comunicacao_ros2/setup.py`

```
entry_points={
    'console_scripts': [
        # ... velocidade_pub, monitor_velocidade,
        # ... modo_server, modo_client
        'controlador = '
        'comunicacao_ros2.controlador:main',
    ],
},
```

Não esqueça

É o quinto executável do pacote `comunicacao_ros2` — a lista de `console_scripts` só cresce, nunca substitui as entradas anteriores.

Compilando e subindo o controlador

colcon build de novo — novo entry_point registrado

```
$ cd ~/tp1_ws  
$ colcon build --packages-select comunicacao_ros2  
$ source install/setup.bash  
$ ros2 run comunicacao_ros2 controlador
```

```
[INFO] [controlador]: nó iniciado
```

Roda sozinho

controlador funciona mesmo sem estado_robo rodando — o serviço responde normalmente, só o callback de /bateria nunca vai disparar (não há quem publique nesse tópico ainda).

Testando /mudar_estado

Dois terminais: chamando o serviço e observando /cmd_vel

Terminal 1 — chamando o serviço

```
$ ros2 service call /mudar_estado \
  std_srvs/srv/SetBool 'data: true'

response:
success=True
message='Estado alterado para movendo'
```

Terminal 2 — observando /cmd_vel

```
$ ros2 topic echo /cmd_vel

linear:
  x: 0.5
angular:
  z: 0.0
```

Teste reverso

Chame de novo com 'data: false' e confirme que /cmd_vel volta para x: 0.0 — é exatamente esse antes/depois que o TP3 pede em prints.

Teste completo: bateria crítica → emergencia

Requer estado_robo.py rodando (Competência 2)

Terminal 1 — estado_robo

```
$ ros2 run comunicacao_ros2 estado_robo  
  
(publica /bateria decrescendo  
de 100.0 até 0.0)
```

Terminal 2 — controlador

```
$ ros2 run comunicacao_ros2 controlador  
  
[WARN] [controlador]:  
BATERIA CRÍTICA: emergencia
```

Resultado esperado

Quando /bateria < 20.0, o controlador loga a transição e /cmd_vel volta a 0.0 automaticamente — mesmo que o robô estivesse 'movendo'.

Inspecionando o serviço

ros2 service list e type — conferindo o registro

```
$ ros2 service list
/controlador/describe_parameters
/mudar_estado

$ ros2 service type /mudar_estado
std_srvs/srv/SetBool
```

Leitura

Se '/mudar_estado' não aparecer na lista, o nó controlador não está rodando ou não terminou de inicializar — confira o terminal onde ele foi executado.

Estado compartilhado sem dor de cabeça

- Todo acesso (leitura OU escrita) a um atributo compartilhado entre callbacks deve passar por um 'with self.lock'
- Mantenha o bloco 'with self.lock' o menor possível — copie o valor para uma variável local e libere o lock antes de operações demoradas
- Nunca chame outro método que também tente adquirir o mesmo lock de dentro de um 'with self.lock' (deadlock)
- Se um nó só tem timers OU só tem um subscriber (nunca os dois disputando o mesmo atributo), SingleThreadedExecutor já é seguro e mais simples

Quando usar cada proteção

Cenário	Precisa de Lock?	Precisa de MultiThreadedExecutor?
Só timers publicando	Não	Não
Serviço + timer, mesmo atributo	Sim	Sim (senão o serviço bloqueia o timer)
Serviço + subscriber + timer (controlador.py)	Sim	Sim
Vários publishers, sem estado compartilhado	Não	Opcional

OK

`threading.Lock`

sei proteger um atributo compartilhado entre callbacks

OK

`/mudar_estado (SetBool)`

controlador.py responde e muda o estado interno corretamente

OK

`MultiThreadedExecutor`

sei quando é obrigatório e como configurar no main()

OK

`/bateria → emergencia`

sei explicar a transição automática e por que não tem volta

Próximo passo

Com o `controlador.py` pronto (Exercício 2 do TP3 coberto), falta a peça que alimenta seu subscriber de bateria: na próxima competência, vamos construir o `estado_robo.py`, que compõe múltiplas mensagens padrão do ROS 2 — `Pose2D`, `Twist`, `Float32` e `String` — para simular o estado completo de um robô móvel.

Criar Mensagens e Serviços usando Python

O que vamos ver

1

O ecossistema de mensagens padrão

`std_msgs`, `geometry_msgs`, `sensor_msgs`, `nav_msgs`

2

Por que compor em vez de criar interface

As vantagens de reaproveitar tipos prontos

3

Publishers múltiplos em um nó

Vários tópicos, um só timer

4

`estado_robo.py`

O nó completo do TP3 Exercício 1

Relembrando: geometry_msgs/Twist

O único tipo padrão que já usamos até agora

- Twist tem dois campos compostos: linear (x,y,z) e angular (x,y,z), todos float64
- Usamos só linear.x e angular.z no velocidade_pub.py do TP2
- Hoje vamos conhecer outros tipos padrão — e usar vários ao mesmo tempo no mesmo nó

O ecossistema de mensagens padrão do ROS 2

Quatro pacotes cobrem a grande maioria dos casos de uso robóticos

std_msgs
tipos primitivos

geometry_msgs
pose e movimento

sensor_msgs
dados de sensores

nav_msgs
mapas e trajetórias

O TP3 Exercício 1 pede para explicar o papel de cada um destes quatro pacotes — os próximos slides fazem exatamente isso.

std_msgs: tipos primitivos

Sem estrutura robótica — só o dado cru, com um campo 'data'

Tipo	Campo(s)	Uso típico no TP3
<code>std_msgs/msg/Float32</code>	float32 data	/bateria — nível de carga (0.0 a 100.0)
<code>std_msgs/msg/String</code>	string data	/status — texto de estado ('operando', 'bateria_critica')
<code>std_msgs/msg/Bool</code>	bool data	flags simples (ex.: /robo_ocupado no AT)
<code>std_msgs/msg/Int32</code>	int32 data	contadores, IDs

geometry_msgs: pose e movimento

Estruturas compostas para posição e velocidade

Tipo	Campos principais	Uso típico no TP3
<code>geometry_msgs/msg/Pose2D</code>	float64 x, y, theta	/pose — posição 2D simplificada do robô
<code>geometry_msgs/msg/Twist</code>	Vector3 linear, Vector3 angular	/velocidade e /cmd_vel — comandos de movimento
<code>geometry_msgs/msg/Point</code>	float64 x, y, z	posições 3D isoladas
<code>geometry_msgs/msg/PoseStamped</code>	Header + Pose (3D)	pose 3D completa, com timestamp e frame

sensor_msgs: dados de sensores

O que os sensores do robô realmente publicam

Tipo	Campos principais	Uso típico
<code>sensor_msgs/msg/Range</code>	float32 range, field_of_view	sensor ultrassônico/infravermelho de distância
<code>sensor_msgs/msg/LaserScan</code>	ranges[], angle_min/max	LiDAR — varredura 2D de distâncias
<code>sensor_msgs/msg/Image</code>	matriz de pixels + encoding	câmera
<code>sensor_msgs/msg/Temperature</code>	float64 temperature	sensor de temperatura ambiente

nav_msgs: mapas e trajetórias

O vocabulário da navegação autônoma

Tipo	Campos principais	Uso típico
<code>nav_msgs/msg/Odometry</code>	Pose + Twist + covariância	posição estimada com incerteza (fusão de sensores)
<code>nav_msgs/msg/Path</code>	PoseStamped[]	sequência de poses — uma rota planejada
<code>nav_msgs/msg/OccupancyGrid</code>	matriz de ocupação	mapa 2D do ambiente

Por que compor mensagens padrão?

O TP3 pede explicitamente: 'sem interfaces customizadas'

- Mensagens padrão já são entendidas por qualquer ferramenta do ecossistema (rqt, rviz, nav2, outros pacotes)
- Não exigem build especial de interfaces (.msg próprios precisam de um pacote de interfaces separado, com regras de build específicas)
- Facilitam integração: qualquer outro nó ROS 2 já sabe interpretar um Twist ou um Float32
- 'Compor o estado' significa: várias mensagens padrão, cada uma em seu próprio tópico, representando aspectos diferentes do mesmo robô

Quando criar uma interface customizada?

Só quando nenhuma combinação de tipos padrão representa bem o dado — por exemplo, um relatório de diagnóstico com muitos campos específicos do seu robô.

Anatomia de Pose2D

Um Pose2D vazio já vem com todos os campos zerados

```
from geometry_msgs.msg import Pose2D

pose = Pose2D()
pose.x = 1.5      # metros
pose.y = 0.0      # metros
pose.theta = 0.0  # radianos
```

Por que Pose2D e não Pose

Pose2D é mais simples: x, y, theta (três float64). O Pose 'completo' de geometry_msgs usa Point (x,y,z) + Quaternion para 3D — desnecessário para um robô que só anda no plano.

Publicando /pose

x incrementando a cada ciclo, como pede o TP3

salvar em `comunicacao_ros2/comunicacao_ros2/estado_robo.py`

```
self.pub_pose = self.create_publisher(  
    Pose2D, '/pose', 10)  
self.x = 0.0  
  
def tick(self):  
    pose = Pose2D()  
    self.x += 0.1  
    pose.x = self.x  
    self.pub_pose.publish(pose)
```

`self.x` como estado

Como não há sensor real, simulamos avanço: um atributo de instância guarda a posição atual e é incrementado a cada chamada do timer, então publicado.

Publicando /velocidade

Velocidade linear constante de 0.3 m/s

salvar em `comunicacao_ros2/comunicacao_ros2/estado_robo.py`

```
self.pub_vel = self.create_publisher(
    Twist, '/velocidade', 10)

def tick(self):
    # ... (continuação do mesmo método)
    vel = Twist()
    vel.linear.x = 0.3
    self.pub_vel.publish(vel)
```

/velocidade vs /cmd_vel

Note o nome do tópico: /velocidade aqui é só um relato de estado (o que o robô está fazendo), diferente de /cmd_vel do controlador.py, que é um comando. São conceitos diferentes, por isso tópicos diferentes.

Publicando /bateria

Decrescendo de 100.0 até 0.0 ao longo do tempo

salvar em `comunicacao_ros2/comunicacao_ros2/estado_robo.py`

```
self.pub_bateria = self.create_publisher(
    Float32, '/bateria', 10)
self.bateria = 100.0

def tick(self):
    # ...
    self.bateria = max(0.0, self.bateria - 2.0)
    msg = Float32()
    msg.data = self.bateria
    self.pub_bateria.publish(msg)
```

`max(0.0, ...)`

O `max()` evita que a bateria fique negativa depois de chegar a zero — sem isso, o valor continuaria decrescendo indefinidamente.

Publicando /status

Texto condicional baseado no nível de bateria

salvar em `comunicacao_ros2/comunicacao_ros2/estado_robo.py`

```
self.pub_status = self.create_publisher(  
    String, '/status', 10)  
  
def tick(self):  
    # ...  
    status = String()  
    status.data = ('bateria_critica'  
        if self.bateria < 20.0  
        else 'operando')  
    self.pub_status.publish(status)
```

Validação do TP3

'ros2 topic echo /status' deve mostrar
'operando' no início e mudar para
'bateria_critica' quando /bateria cruzar 20.0
— é essa transição que o TP3 pede
evidenciada em print.

Um nó, quatro publishers, um timer

Todos disparados pelo mesmo tick(), a cada 1 segundo



Um só `create_timer(1.0, self.tick)` — dentro do `tick()`, as quatro publicações acontecem em sequência, na mesma thread.

estado_robo.py — parte 1 de 3

Imports e construtor: quatro publishers e o estado simulado

salvar em `comunicacao_ros2/comunicacao_ros2/estado_robo.py`

```
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Pose2D, Twist
from std_msgs.msg import Float32, String
class EstadoRobo(Node):
    def __init__(self):
        super().__init__('estado_robo')
        self.pub_pose = self.create_publisher(Pose2D, '/pose', 10)
        self.pub_vel = self.create_publisher(Twist, '/velocidade', 10)
        self.pub_bateria = self.create_publisher(Float32, '/bateria', 10)
        self.pub_status = self.create_publisher(String, '/status', 10)
        self.x = 0.0
        self.bateria = 100.0
        self.create_timer(1.0, self.tick)
```

estado_robo.py — parte 2 de 3

tick(): pose, velocidade e bateria

salvar em `comunicacao_ros2/comunicacao_ros2/estado_robo.py`

```
def tick(self):
    pose = Pose2D()
    self.x += 0.1
    pose.x = self.x
    self.pub_pose.publish(pose)
    vel = Twist()
    vel.linear.x = 0.3
    self.pub_vel.publish(vel)
    self.bateria = max(0.0, self.bateria - 2.0)
    bat_msg = Float32()
    bat_msg.data = self.bateria
    self.pub_bateria.publish(bat_msg)
```

estado_robo.py — parte 3 de 3

tick(): status condicional, e o main()

salvar em `comunicacao_ros2/comunicacao_ros2/estado_robo.py`

```
status = String()
status.data = ('bateria_critica'
               if self.bateria < 20.0 else 'operando')
self.pub_status.publish(status)
```

```
def main(args=None):
    rclpy.init(args=args)
    rclpy.spin(EstadoRobo())
    rclpy.shutdown()
```

Registrando o `entry_point`

`estado_robo` precisa entrar no `setup.py`

salvar em `comunicacao_ros2/setup.py`

```
entry_points={
    'console_scripts': [
        # ... velocidade_pub, monitor_velocidade,
        # ... modo_server, modo_client, controlador
        'estado_robo = '
        'comunicacao_ros2.estado_robo:main',
    ],
},
```

Sexto executável

O pacote `comunicacao_ros2` já hospeda seis nós diferentes — é normal um pacote crescer assim ao longo de um projeto real.

Compilando e subindo o estado_robo

colcon build + ros2 run

```
$ cd ~/tp1_ws  
$ colcon build --packages-select comunicacao_ros2  
$ source install/setup.bash  
$ ros2 run comunicacao_ros2 estado_robo
```

```
[INFO] [estado_robo]: nó iniciado
```

Deixe rodando

Mantenha esse terminal aberto — é esse processo que alimenta /bateria para o teste completo do controlador.py, feito na Competência 1.

Conferindo os quatro tópicos

ros2 topic list -t — critério de validação do TP3

```
$ ros2 topic list -t
/pose [geometry_msgs/msg/Pose2D]
/velocidade [geometry_msgs/msg/Twist]
/bateria [std_msgs/msg/Float32]
/status [std_msgs/msg/String]
```

No TP3

O Exercício 1 pede exatamente essa lista de quatro tópicos com seus tipos — cole esse print no relatório.

Observando a transição de status

ros2 topic echo /status — evidência pedida no TP3

```
$ ros2 topic echo /status
data: operando
---
data: operando
---
data: bateria_critica
---
```

Quanto tempo esperar

Com bateria caindo 2.0/s e limiar em 20.0, a transição acontece por volta dos 40s de execução (100 → 20 em 40 ciclos de 1s).

Os quatro tópicos do estado_robo.py

A mesma tabela que organiza a validação do Exercício 1

Tópico	Tipo	Comportamento
/pose	geometry_msgs/msg/Pose2D	x incrementa 0.1 a cada ciclo
/velocidade	geometry_msgs/msg/Twist	linear.x = 0.3 constante
/bateria	std_msgs/msg/Float32	decrece 2.0/ciclo, piso em 0.0
/status	std_msgs/msg/String	'operando' → 'bateria_critica' abaixo de 20.0

Organizando múltiplos publishers

- Um atributo `self.pub_<algo>` por publisher — nomes claros evitam confundir qual publisher é qual
- Centralize toda a lógica de simulação em um único callback de timer, como o `tick()` — mais fácil de auditar que espalhar em vários timers
- Guarde estado simulado (`self.x`, `self.bateria`) como atributos de instância, nunca como variáveis globais
- Escolha o tipo padrão mais simples que representa o dado — `Pose2D` em vez de `PoseStamped` quando não precisar de `timestamp/frame`

OK

`std_msgs / geometry_msgs /
sensor_msgs / nav_msgs`

sei explicar o papel de cada pacote

OK

`Pose2D, Twist, Float32, String`

estado_robo.py publica os quatro simultaneamente

OK

`ros2 topic list -t`

consigo conferir os quatro tópicos e seus tipos

OK

`bateria < 20.0 → status`

sei demonstrar a transição de 'operando' para 'bateria_critica'

Você está pronto para o TP3 (parcialmente)

Com `estado_robo.py` e `controlador.py` prontos, os Exercícios 1 e 2 do TP3 já estão cobertos — e o teste completo de bateria crítica → emergência (visto na Competência 1) já pode ser reproduzido de ponta a ponta. Na Etapa 6, vamos implementar o Exercício 3 (ação ROS 2 com `patrulha_server/patrulha_client`) e o Exercício 4 (o sistema de vigilância que integra tópicos, serviços e ações).